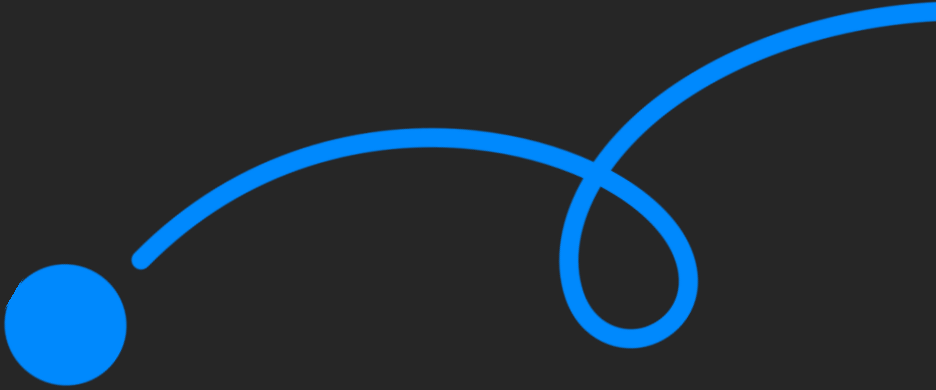




코멘토 실무PT 클래스

반도체 SW 엔지니어를 위한 새로운 SoC 위에 리눅스 디바이스 드라이버 포팅하기

멘티님, 반갑습니다!

클래스는 20:00에 시작됩니다.

출석 확인을 위해 프로필은 **실명**으로 설정해주세요.

출석을 확인합니다.



김대용

김진영

원종진

조현경

최승원

최민우

최강주

지난 세션은 어떠셨나요.

- 1 지난 주차에 어떤걸 공부했었는지 짧게 리뷰 해봐요.
- 2 복습을 진행하면서 궁금한 점이 있다면 같이 토의해봐요.
- 3 과제를 진행하면서 어려웠던 점을 알려주세요.
- 4 멘티 여러분이 작성한 과제를 다같이 살펴보도록 해요.

시작하기에 앞서

1

수업 진행을 하며
멘티님들의
이해도를 체크하며
진행할 예정입니다.

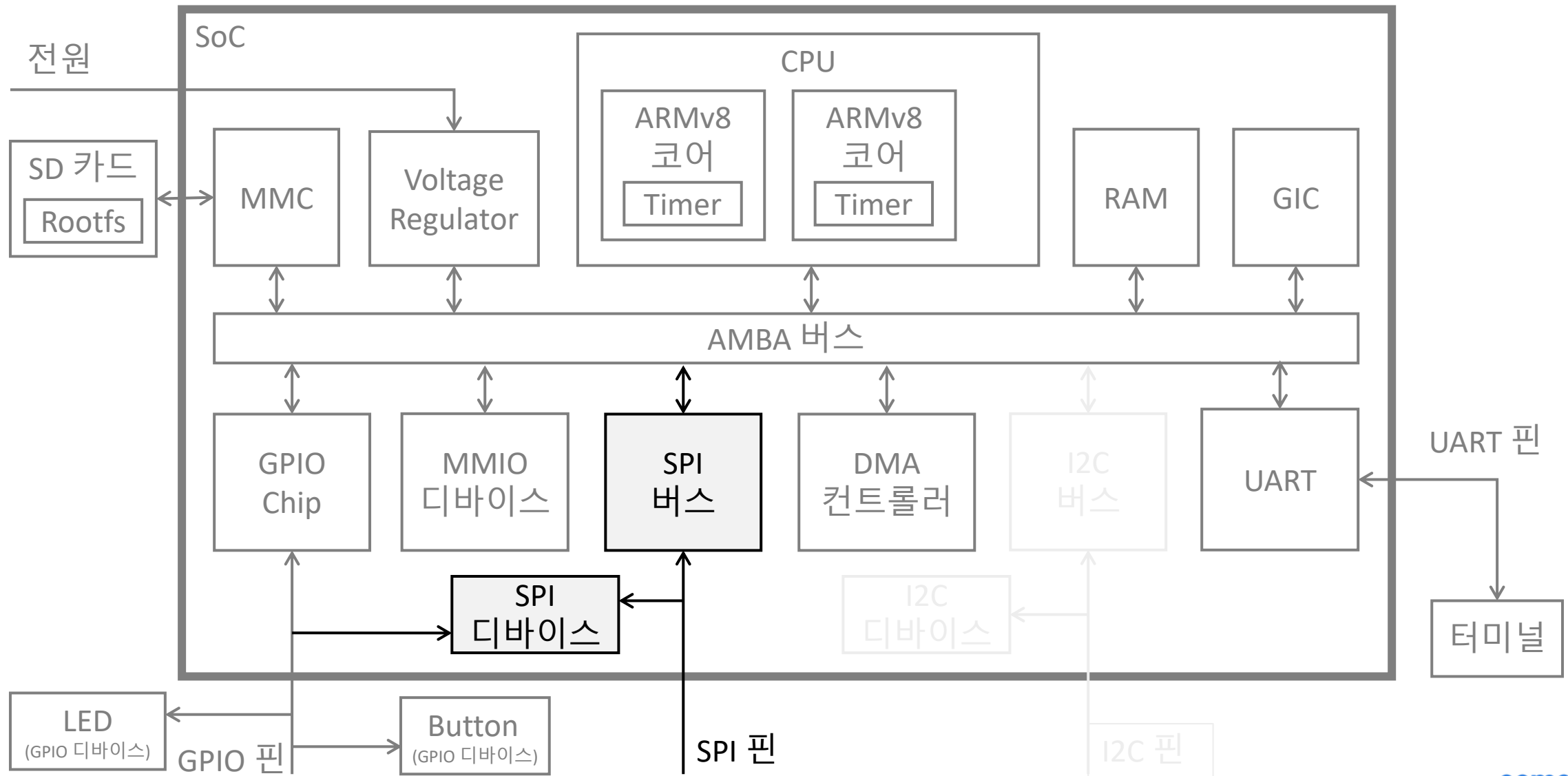
2

이해가 되지 않거나,
궁금한 내용은
채팅으로 언제든지
남겨주세요.

3

평균 50분 라이브를
진행하고, 10분의
쉬는 시간으로
진행 됩니다.

이번 수업에서 진행할 내용



01 SPI 통신 이해하기

02 새로운 SPI 하드웨어 만들기

03 SPI 툴로 SPI 하드웨어 테스트하기

04 SPI 하드웨어 드라이버 만들기

05 ADC 이해하기

Chapter. 1

SPI 통신 이해하기

SPI (Serial Peripheral Interface) 란?

SoC와 주변기기를 연결하기 위한 인터페이스

- SoC 외부와의 연결을 위해 사용하나 SoC 내부에서 간단한 통신을 위해 사용되기도 함
- 1 (Master) : N (Slave) 의 통신 지원
 - SoC를 Master로 주변기기들을 Slave로 둠

하드웨어적 구현이 비교적으로 간단하므로 많이 사용됨

본 강의에서 배우는 칩 외부통신 방법 중 가장 빠른 방식

- UART (~115kbps) < 다음주에 다룰 I2C (100kbps~3.4Mbps) < SPI (10Mbps ~)

SSI (Synchronous Serial Interface)라고도 함

- 단, SSI는 1:1의 장거리 통신에 더 초점

SPI 구조

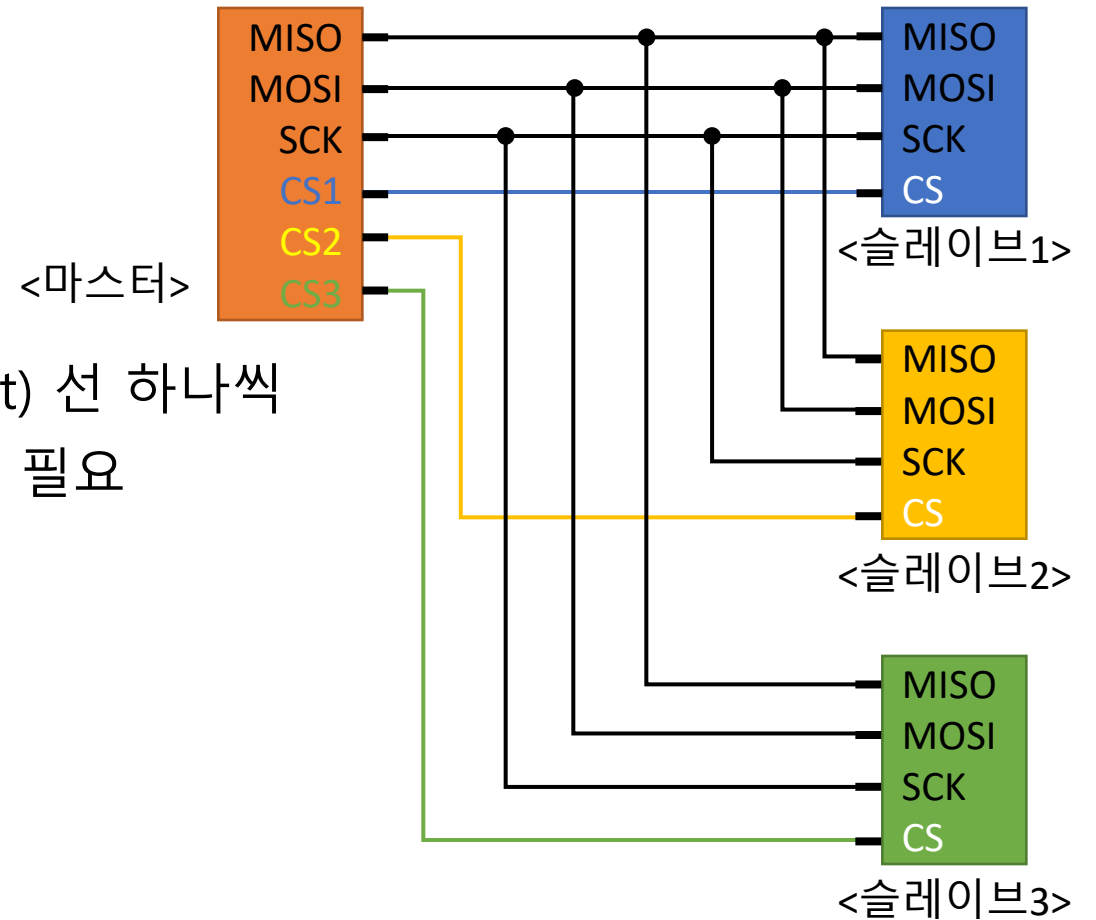
SoC와 주변기기의 3개의 신호선 끼리 연결 필요

- 송신 (MOSI, Master-Out-Slave-In)
- 수신 (MISO, Master-In-Slave-Out)
- 클럭 (SCK)

주변기기는 슬레이브로 각각 칩선택(CS, Chip Select) 선 하나씩
SoC는 마스터로 모든 주변기기의 칩선택 선과 연결 필요

SoC와 주변기기의 VCC 전압은 같아야 함

SoC와 주변기기의 GND 끼리 연결 필요



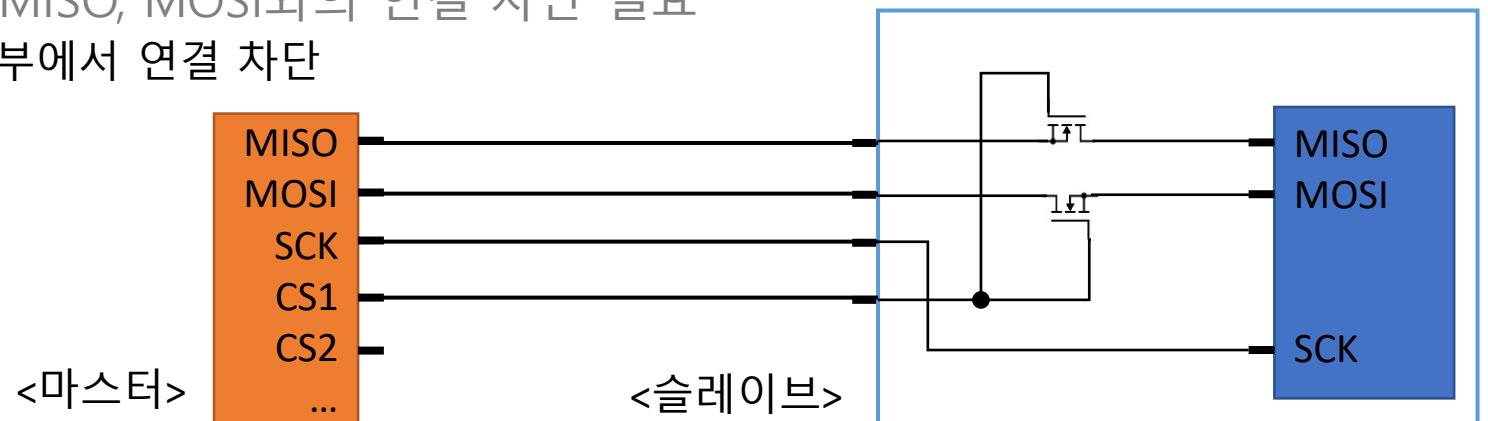
SPI 단점

데이터 송수신을 위해 MISO, MOSI를 여러 주변장치가 공유

- 동시에 여러 주변장치가 신호선을 사용할 때 충돌 발생 가능
 - I2C와는 달리 충돌을 해결하기 위한 공식적인 방법은 없음

한번에 한 주변장치만 MISO, MOSI 사용하도록 칩선택 신호 필요

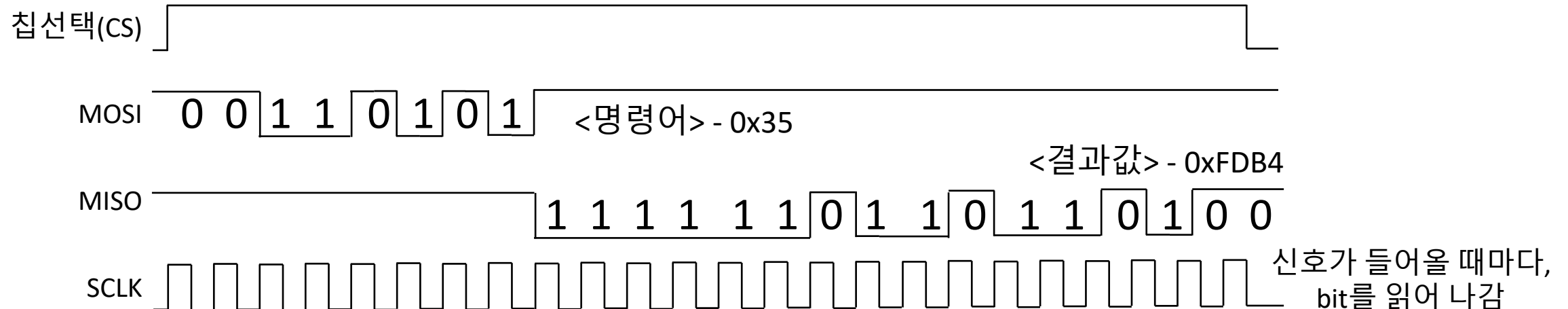
- 주변장치를 추가할 때마다 칩선택 신호를 위한 GPIO 추가 할당 필요
- 칩선택 신호에 따라 물리적으로 MISO, MOSI와의 연결 차단 필요
 - FET를 스위치로 사용하여 칩 내부에서 연결 차단



SPI 통신 흐름

마스터의 SCK의 클럭에 따라 MISO / MOSI에 1비트씩 쓰거나 읽어옴

- 명확하게 정해진 프로토콜은 없으나 일반적으로 다음 흐름을 따름
 1. 마스터가 칩선택 선의 신호를 High로 바꿈
 2. 마스터가 MOSI에 원하는 동작을 위해 명령어를 바이트 단위로 보냄
 3. 슬레이브는 명령어에 해당하는 동작 수행
 4. 슬레이브는 동작에 대한 결과값을 MISO로 바이트 단위로 보냄



실제 SoC에서 SPI의 형태

라즈베리 파이에 사용되는 SoC(Broadcom社 BCM27xx)는 SPI 연결을 지원

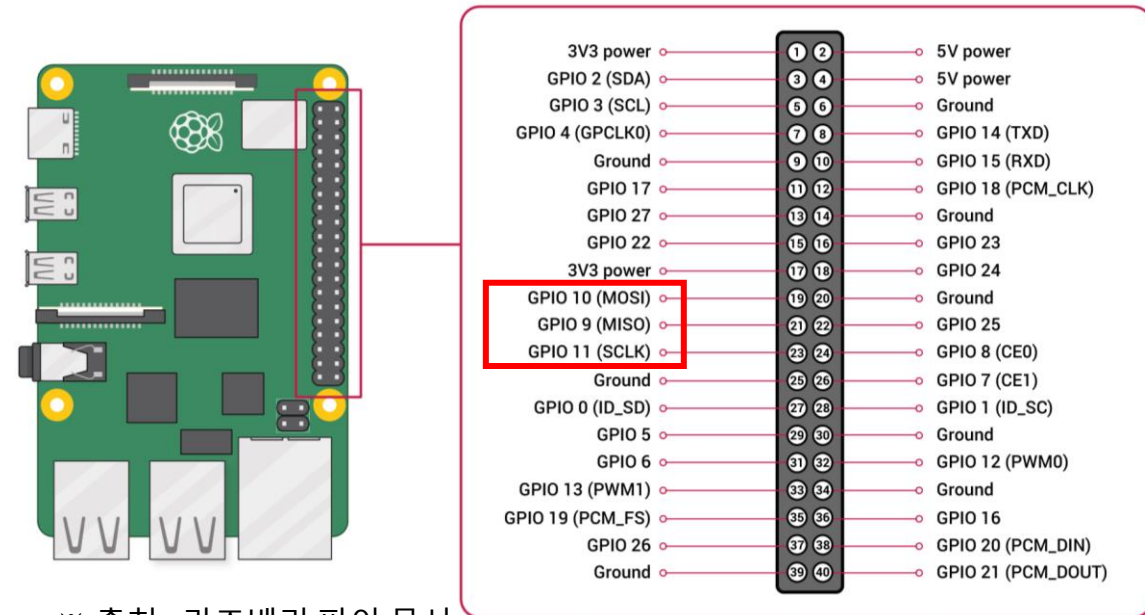
- SPI를 사용하는 다양한 형태의 주변기기 (센서, 메모리, LCD 등) 연결 가능

외부로 나온 핀 헤더를 사용하여 SPI 사용 가능

- GPIO와 겸용으로 사용 되는 핀
 - SPI로 사용하도록 사전 설정 필요

주변장치 여러 개라면, GPIO를 칩선택으로 사용

- 통신하고 싶은 주변 장치의 칩선택만 High로 설정



※ 출처 : 라즈베리 파이 문서

SPI를 사용하는 주변기기 예시

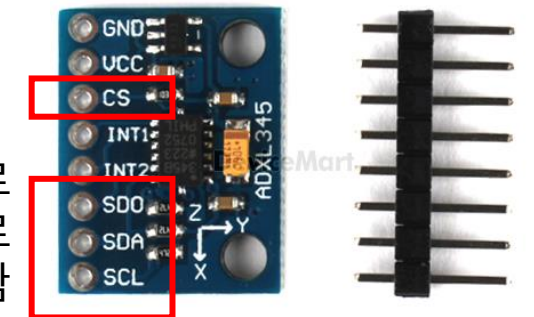
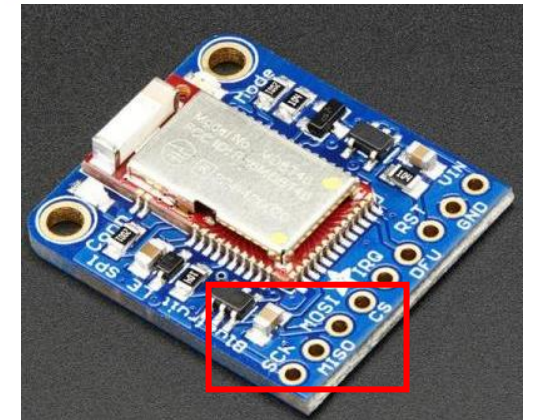
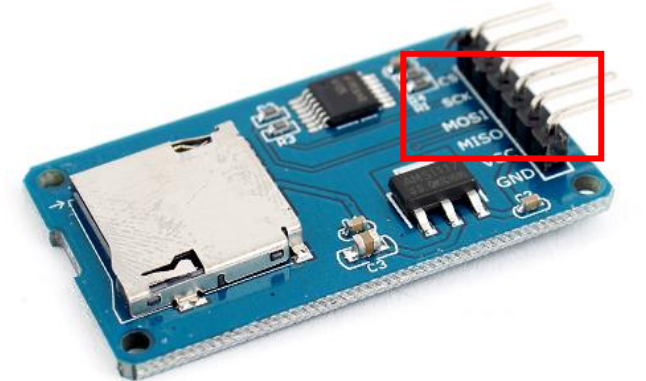
상대적으로 빠른 속도를 요구하는 주변기기

- SoC에 MMC 컨트롤러가 없어도, SD 카드 자체를 SPI로 사용 가능
 - SD카드에서 SPI 통신을 자체적으로 지원
 - 예시 : <https://www.devicemart.co.kr/goods/view?no=1326951>
- 간단하게 붙일 수 있는 형태의 블루투스, WIFI 등의 통신 모듈
 - 예시 : <https://www.devicemart.co.kr/goods/view?no=12518678>

많은 센서들이 I2C와 SPI 통신 둘다 지원

- 가속도 센서, 온도도 센서 등
 - 예시 : <https://www.devicemart.co.kr/goods/view?no=1314305>

높은 프레임을 요구하지 않는 LCD 디스플레이



MISO를 SDO로
MOSI를 SDA로
표기하기도 함



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

Chapter. 2

새로운 SPI 하드웨어 만들기

SPI 컨트롤러란?

SPI 통신은 SPI 컨트롤러라는 별도의 하드웨어를 사용

- 개발자는 SPI 통신의 구체적인 구현을 신경 쓸 필요가 없음

SoC 외부와 통신해야 하므로 물리적인 특성을 고려해야 함

- SCK 클럭에 맞추어 1비트씩 MOSI / MISO 핀으로 데이터 송수신
 - MISO 핀으로 온 1비트씩 수신된 데이터를 1바이트로 조합
 - 한번에 1바이트의 데이터를 MOSI 핀으로 1비트씩 전송

QEMU의 새 SoC에 SPI 컨트롤러 추가하기

```
...
#include "hw/ssi/ssi.h"
...
struct ComentoMachineState {
    ...
    DeviceState *spi;
};
...
#define COMENTO_IRQ_SPI      5
...
#define COMENTO_BASE_SPI     0x09015000
...
static void create_spi(ComentoMachineState *vms, MemoryRegion *mem) {
    hwaddr base = COMENTO_BASE_SPI;
    int irq = COMENTO_IRQ_SPI;
    SysBusDevice *s;
    char name[] = "spi";

    // ARM PL022 SPI 컨트롤러 장치 추가
    vms->spi = qdev_new("pl022");

    // QMP 스크립트에서 SPI 버스를 위해 사용할 이름 지정
    qdev_set_id(vms->spi, name, NULL);

    s = SYS_BUS_DEVICE(vms->spi);
    sysbus_realize_and_unref(s, &error_fatal);
    memory_region_add_subregion(mem, base, sysbus_mmio_get_region(s, 0));
    sysbus_connect_irq(s, 0, qdev_get_gpio_in(vms->gic, irq));
}
```

```
...
static void machcomento_init(MachineState *machine)
{
    ...
    create_spi(vms, sysmem);
}
```

<hw/arm/comento.c>

디바이스 트리와 커널 설정에 SPI 컨트롤러 추가하기

```
...
spi@9015000 {
    compatible = "arm,pl022", "arm,primecell";
    // SPI 컨트롤러의 인터럽트와 MMIO 주소 명시
    interrupts = <GIC_SPI 5 IRQ_TYPE_LEVEL_HIGH>;
    reg = <0x00 0x9015000 0x00 0x1000>;
    // AMBA 장치이므로 클럭 필요
    clock-names = "apb_pclk";
    clocks = <&clock>;
    // SPI 디바이스를 명시할 때 숫자는 1칸 사용
    #address-cells = <1>;
    // SPI 디바이스를 명시할 때 크기는 필요 없으므로 0칸으로 지정
    #size-cells = <0>;

    // 칩선택으로 사용하려는 GPIO 핀의 개수 명시
    num-cs = <3>;

    // 칩선택으로 사용하려고 하는 GPIO 핀 명시
    // 슬레이브가 GPIO 핀을 칩선택으로 사용하지 않는다면, 0으로 지정
    cs-gpio = <&gpio 4 IRQ_TYPE_LEVEL_HIGH>,
              <&gpio 5 IRQ_TYPE_LEVEL_HIGH>,
              <0>;
};
...
```

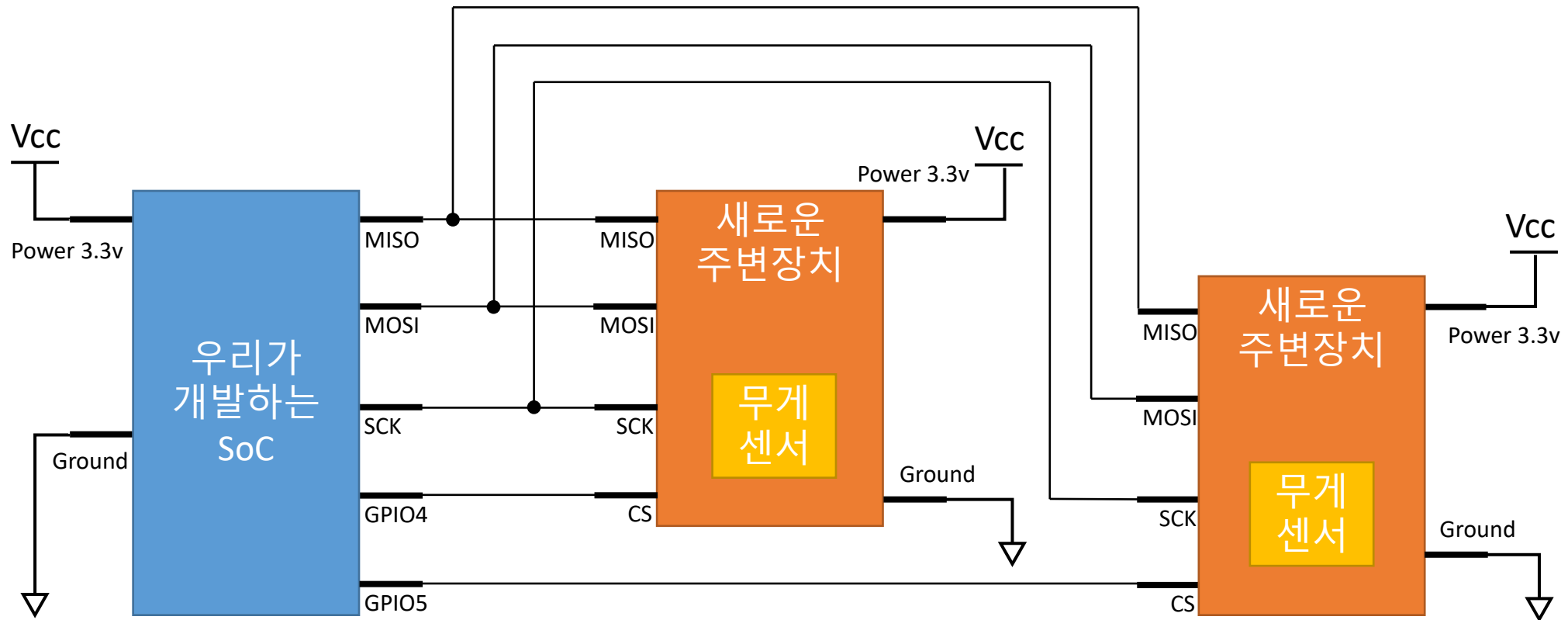
<arch/arm64/boot/dts/comento/comento.dts>

```
...
CONFIG_SPI=y
CONFIG_SPI_PL022=y
# SPI 톨을 사용하기 위해 필요한 설정
# SPI 톨을 사용하기 위해서는 spidev를 사용해야 함
CONFIG_SPI_SPIDEV=y
```

<arch/arm64/configs/comento_defconfig>

수정한 defconfig를 적용하기 위해
make comento_defconfig 명령어 사용

수업에서 사용할 SPI의 시나리오



- 위 회로와 비슷하게 SPI 사용 하드웨어를 QEMU로 에뮬레이션 해봅시다
 - GPIO4, GPIO5를 칩선택으로 사용하여 주변장치 2개 추가
 - 주변장치는 무게(QMP의 scale 속성으로 대체) 센서로 현재 무게 읽기, 0점 조정하기의 2가지 기능
 - 마스터가 먼저 1바이트의 명령어 코드를 보내면, 주변장치는 2바이트의 결과 값을 반환

QEMU에 SPI를 사용하는 주변장치 추가하기 (1/2)

```
#include "qemu/osdep.h"
#include "hw/sysbus.h"
#include "hw/irq.h"
#include "hw/ssi/ssi.h"
#include "sysemu/runstate.h"
#include "qapi/visitor.h"
#define TYPE_SSI_COMENTO "ssi-comento"
#define COMENTO_SSI_GET_SCALE 1 // 명령어 코드를 2개 정의 :
#define COMENTO_SSI_SET_ZERO 2 // 현재 무게 얻어오기, 영점 조정하기

OBJECT_DECLARE_SIMPLE_TYPE(SSICOMENTO_State, SSI_COMENTO)
struct SSI_COMENTO_State {
    SSIPeripheral parent_obj;
    uint16_t scale;
    uint16_t offset;
    int cycle;
};
// SPI 통신을 처리하기 위한 함수
// data 파라미터로는 MOSI의 입력, 반환 값은 MISO로 출력
// 명령어 코드로 1바이트, 반환 값으로 2바이트를 사용하는 구현
static uint32_t ssi_comento_transfer(SSIPeripheral *obj, uint32_t data)
{
    SSI_COMENTO_State *s = SSI_COMENTO(obj);
    if (data != 0) {
        // 영점 조정을 위해 COMENTO_SSI_SET_ZERO 명령어를 보낸 경우
        if (data == COMENTO_SSI_SET_ZERO) {
            s->offset = s->scale; // 영점을 현재 무게 값으로 조정
        }
        s->cycle = 0; // 명령어를 뭔가 받았다면 현재 cycle을 0번으로 설정
    } else {
        s->cycle++; // 명령어를 받지 않았다면 cycle 증가
    }
}
```

```
if (s->cycle == 1) { // 1번 사이클이면 scale의 상위 8비트 반환
    return (uint8_t)((s->scale - s->offset) >> 8);
} else if (s->cycle == 2) { // 2번 사이클이면 scale의 하위 8비트 반환
    return (uint8_t)(s->scale - s->offset);
}
return 0; // 0번 사이클에는 데이터를 0으로 보냄
}
// QMP로 scale 속성을 얻어오려고 할 때 문자열로 반환
static char *ssi_comento_get_scale(Object *obj, Error **errp)
{
    SSI_COMENTO_State *s = SSI_COMENTO(obj);
    char buf[8];
    sprintf(buf, "%d\n", s->scale);
    return g_strdup(buf);
}
// QMP로 scale 속성을 문자열로 넣었을 때 값 갱신
static void ssi_comento_set_scale(Object *obj, const char *value,
                                   Error **errp)
{
    SSI_COMENTO_State *s = SSI_COMENTO(obj);
    sscanf(value, "%hd", &s->scale);
}

static void ssi_comento_init(Object *obj)
{
    object_property_add_str(obj, "scale",
                             ssi_comento_get_scale, ssi_comento_set_scale);
}
// 버스에 연결되자마자 해야 할 일은 없으므로 빈 함수
static void ssi_comento_realize(SSIPeripheral *obj, Error **errp)
{
}
```

QEMU에 SPI를 사용하는 하드웨어 추가하기 (2/2)

```
static void ssi_comento_class_init(ObjectClass *klass, void *data)
{
    SSIPeripheralClass *k = SSI_PERIPHERAL_CLASS(klass);

    k->cs_polarity = SSI_CS_HIGH; // 칩선택이 High 일 때 동작하도록 설정
    k->realize = ssi_comento_realize;
    k->transfer = ssi_comento_transfer;
}

static const TypeInfo ssi_comento_info = {
    .name          = TYPE_SSI_COMENTO,
    .parent        = TYPE_SSI_PERIPHERAL,
    .instance_size = sizeof(SSI_COMENTO_State),
    .instance_init = ssi_comento_init,
    .class_init    = ssi_comento_class_init,
};

static void ssi_comento_register_types(void)
{
    type_register_static(&ssi_comento_info);
}

type_init(ssi_comento_register_types)
```

<hw/misc/comento/ssi.c>

```
...
softmmu_ss.add(when: 'CONFIG_COMENTO', if_true: files('ssi.c'))
```

<hw/misc/comento/meson.build>

```
static void create_spi_devs(ComentoMachineState *vms) {
    DeviceState *dev;
    void *ssi = qdev_get_child_bus(vms->spi, "ssi");
    // GPIO 4번핀과 주변장치의 칩선택핀과 연결
    dev = ssi_create_peripheral(ssi, "ssi-comento");
    qdev_connect_gpio_out(vms->gpio, 4,
                          qdev_get_gpio_in_named(dev, SSI_GPIO_CS, 0));
    // GPIO 5번핀과 주변장치의 칩선택핀과 연결
    dev = ssi_create_peripheral(ssi, "ssi-comento");
    qdev_connect_gpio_out(vms->gpio, 5,
                          qdev_get_gpio_in_named(dev, SSI_GPIO_CS, 0));
}

...
static void machcomento_init(MachineState *machine)
{
    ...
    create_spi_devs(vms);
}
```

<hw/arm/comento.c>

1. hw/misc/comento/에 SPI를 사용하는 "ssi-comento" 하드웨어 추가
2. hw/arm/comento.c에 해당 하드웨어를 추가하고 SPI 컨트롤러의 SPI 버스와 연결

Rootfs에 SPI 툴 추가하기

SPI 버스에 연결된 주변장치를 사용하기 위해서는 2가지 방법

- SPIDEV 디바이스 노드를 사용하는 SPI툴 사용 (디버깅 용도 권장)
- 주변 장치 전용의 디바이스 드라이버를 구현

SPI 툴은 SPIDEV 사용을 위해 만들어진 명령어 도구

- <https://github.com/cpb-/spi-tools/>

Rootfs에 SPI 툴을 추가할 수 있도록 빌드루트 설정

- make menuconfig
 - Target packages -> Hardware handling -> spi-tools : 선택

빌드 루트 빌드하기

- make -j<코어 개수>

Roofs를 SD 카드 이미지로 복사하기 (1/2)

1. 이미지 파일을 Loop 디바이스로 설정

- 이미지 파일을 바로 마운트 불가, Loop 디바이스 상태에서 마운트 가능
- `sudo losetup -Pf --show sdcard.img`

```
mentoring@comento:~$ sudo losetup -Pf --show sdcard.img  
/dev/loop8
```

- 출력되는 디바이스 노드 파일이 설정된 loop 디바이스

2. Loop 디바이스의 첫 번째 파티션을 ext4 형식으로 포맷

- `sudo mkfs.ext4 <loop 디바이스 경로>p1`

```
mentoring@comento:~$ sudo mkfs.ext4 /dev/loop8p1  
mke2fs 1.46.5 (30-Dec-2021)  
Discarding device blocks: done  
Creating filesystem with 16128 4k blocks and 16128 inodes  
  
Allocating group tables: done  
Writing inode tables: done  
Creating journal (1024 blocks): done  
Writing superblocks and filesystem accounting information: done
```

Rootfs를 SD 카드 이미지로 복사하기 (2/2)

3. 포맷한 파티션과 빌드루트에서 생성된 이미지 동시에 마운트
 - `mkdir mnt1 mnt2`
 - `sudo mount -o loop <loop 디바이스 경로>p1 mnt1`
 - `sudo mount -o loop <빌드루트 디렉토리>/output/images/rootfs.ext4 mnt2`
4. 마운트된 빌드루트 이미지의 내용을 마운트된 포맷 파티션으로 복사
 - `sudo cp -R mnt2/* mnt1/.`
5. 마운트한 디렉토리를 모두 언마운트
 - `sync; sudo umount mnt1 mnt2`
6. Loop 디바이스 해제
 - `sudo losetup -d <loop 디바이스 경로>`



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 3

SPI 툴로 SPI 하드웨어 테스트하기

디바이스 트리에 SPIDEV 추가하기

```
...
spi@9015000 {
    ...
    spidev@0 {
        // 리눅스에서는 SPIDEV를 사용하는 것을 추천하지 않음
        // (주변장치를 위한 전용의 드라이버를 만드는 것을 추천함)
        // SPIDEV를 사용해야 하는 주변장치 중 아무거나 하나를 골라
        // compatible에 명시해야 SPIDEV 사용 가능
        compatible = "lwn,bk4";
        spi-max-frequency = <1000000>; // 최대 1Mbps로 SPI의 속도 지정
        // 0번 디바이스임을 명시
        // cs-gpio의 0번째 항목(GPIO 4번 핀)이 칩선택으로 사용됨
        reg = <0>;
    };
    spidev@1 {
        compatible = "lwn,bk4";
        spi-max-frequency = <1000000>;
        // 1번 디바이스임을 명시
        // cs-gpio의 1번째 항목(GPIO 4번 핀)이 칩선택으로 사용됨
        reg = <1>;
    };
};
...
```

<arch/arm64/boot/dts/comento/comento.dts>

SPI 툴을 사용하기 위해서는 반드시 SPIDEV 디바이스 노드 필요

- 디바이스 노드를 위해서는 디바이스 트리에 SPIDEV 디바이스를 추가하고 compatible 명시

SPI 툴 사용하여 SPI 테스트 하기

SPIDEV 디바이스 노드를 사용하여 SPI 통신 수행

- spi-config : SPIDEV를 설정하거나 설정되어 있는 값 얻어옴
 - -d : SPIDEV 지정 (ex> /dev/spidev/0.0)
 - -q : 현재의 모든 설정 값을 얻어옴
 - -b : SPIDEV에 데이터를 읽고 쓸 때, 몇비트 단위로 할지 지정 (기본값 : 8bit = 1바이트)
- spi-pipe : SPIDEV로 데이터 전송
 - 표준 입력으로 데이터를 받아서 SPIDEV로 송신
 - SPIDEV에서 수신된 데이터를 표준 출력으로 출력

SPI 툴 사용하여 SPI 테스트 실습

```
# ls -lah /dev/spidev0*
crw----- 1 root    root      153,  0 Jan  1 00:00 /dev/spidev0.0
# spi-config -d /dev/spidev0.0 -q
/dev/spidev0.0: mode=0, lsb=0, bits=8, speed=1000000, spiready=0
# printf "\x1\x0\x0" | spi-pipe -d /dev/spidev0.0 -n -1 | hexdump -C
00000000  00 00 00                                [...]
00000003
```

```
# printf "\x1\x0\x0" | spi-pipe -d /dev/spidev0.0 -n -1 | hexdump -C
00000000  00 00 19                                [...]
00000003
# printf "\x2" | spi-pipe -d /dev/spidev0.0 -n -1 | hexdump -C
00000000  00                                [...]
00000001
# printf "\x1\x0\x0" | spi-pipe -d /dev/spidev0.0 -n -1 | hexdump -C
00000000  00 00 00                                [...]
00000003
```

```
# printf "\x1\x0\x0" | spi-pipe -d /dev/spidev0.0 -n -1 | hexdump -C
00000000  00 00 07                                [...]
00000003
```

- 영점이 조정되어 32kg를 넣었음에도 7kg로 반환

printf로 3개 바이트를 spi-pipe의 표준입력으로 보냄

- 첫 바이트 : 명령어 코드 , 나머지 바이트 : 반환 값 2바이트를 받기 위해 빈 값으로 사이클 생성

hexdump로 spi-pipe의 표준 출력 내용을 읽을 수 있는 형태로 출력

✓ qom-set 사용시 숫자 뒤에 문자를 붙여야 함 -> QEMU 버그!

- Scale의 초기값은 0이므로 SPI통신으로 0을 받음

```
comento@coment:~/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock
machine/peripheral/spi/ssi/child[0].scale 25kg
{}
```

- 0x02는 현재 scale을 영점으로 하라는 명령어 코드

```
comento@coment:~/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock
machine/peripheral/spi/ssi/child[0].scale 25kg
{}
comento@coment:~/qemu/scripts/qmp$ ./qom-get --socket /tmp/qmp.sock
machine/peripheral/spi/ssi/child[0].scale
25
```

```
comento@coment:~/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock
machine/peripheral/spi/ssi/child[0].scale 32kg
{}
```

Chapter. 4

SPI 하드웨어 드라이버 만들기

리눅스에 새로운 SPI 드라이버 추가하기

1. 기존에 추가했던 drivers 밑의 comento 디렉토리를 그대로 사용
2. drivers/comento/Makefile에 우측과 같이 spi 드라이버 추가
3. 디바이스 트리의 SPIDEV의 compatible을 우측과 같이 새로 추가할 "comento-spi"로 변경

```
obj-y += spi.o
```

```
<drivers/comento/Makefile>
```

```
<arch/arm64/boot/dts/comento/comento.dts>
```

```
...
    spi@9015000 {
        ...
        spidev@0 {
            ...
            compatible = "comento-spi";
            ...
        };
        spidev@1 {
            ...
            compatible = "lwn,bk4";
            ...
        };
    };
...
```


리눅스에 새로운 SPI 드라이버 추가하기 (1/2)

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/device.h>
#include <linux/spi/spi.h> // SPI 관련 API를 사용하기 위해 include 함

struct comento_spi_device {
    struct spi_device *dev;
    struct spi_message msg; // 어떤 식으로 통신할 지 미리 정해두어야 함
    struct spi_transfer xfer[2]; // 메시지에 포함 될 전송 방법
    char tx_buf[1]; // 송신용 데이터 버퍼
    char rx_buf[2]; // 수신용 데이터 버퍼
    //송신할 데이터는 명령어코드 1바이트, 수신할 데이터는 반환값 2바이트
    struct mutex lock;
};

// SPI 주변장치가 디바이스 트리에 명시되어 탐지 되었을 때 호출 됨
static int comento_spi_probe(struct spi_device *dev)
{
    struct comento_spi_device *csdev;
    int err;

    dev->bits_per_word = 8; // 주변장치는 한번에 1바이트씩 통신
    dev->mode = SPI_MODE_0; // 주변장치가 사용할 SPI 처리 방법
    // SPI_MODE_2은 SCK 클럭의 라이징 엣지에 비트를 처리함을 의미
    // 하드웨어 구현에 따라 다른 부분 (QEMU에서는 신경 쓰지 않아도 됨)
    err = spi_setup(dev);
    if (err < 0) return err;

    csdev = devm_kzalloc(&dev->dev, sizeof(struct comento_spi_device),
                        GFP_KERNEL);
    if (csdev == NULL) return -ENOMEM;
```

```
// 어떤식으로 통신할지 명시하는 메시지 구조체를 초기화
spi_message_init(&csdev->msg);

csdev->xfer[0].tx_buf = csdev->tx_buf; // 처음에는 송신버퍼만 사용
csdev->xfer[0].len = 1; // 송신 크기는 1바이트
// 메시지에 전송방법 추가
spi_message_add_tail(&csdev->xfer[0], &csdev->msg);

csdev->xfer[1].rx_buf = csdev->rx_buf; // 처음에는 수신버퍼만 사용
csdev->xfer[1].len = 2; // 수신 크기는 2바이트
// 메시지에 전송방법 추가
spi_message_add_tail(&csdev->xfer[1], &csdev->msg);

// 메시지에 2가지 전송 방법이 추가 된 것 :
// 첫번째는 송신 버퍼의 1바이트 송신,
// 두번째는 수신 버퍼로 2바이트 수신

mutex_init(&csdev->lock);

csdev->dev = dev;
// 나중에 spi_get_drvdata로 얻어오기 위해 추가
spi_set_drvdata(dev, csdev);

return 0;
}

// 드라이버 모듈이 언로드 되면 호출 됨
static void comento_spi_remove(struct spi_device *dev)
{
    struct comento_spi_device *csdev = spi_get_drvdata(dev);
    mutex_destroy(&csdev->lock);
}
```

리눅스에 새로운 SPI 드라이버 추가하기 (2/2)

```
static const struct spi_device_id comento_spi_ids[] = {
    { "comento-spi", 0 }, // 이름을 여러 개 명시도 가능
    { },
}; // 디바이스트리의 compatible에서 사용할 드라이버의 이름 명시

// SPIDEV를 초기화할 때 compatible에 comento_spi_ids에 지정한
// 이름이 있다면 이 드라이버를 초기화 하도록 설정
MODULE_DEVICE_TABLE(spi, comento_spi_ids);

static struct spi_driver comento_spi_driver = {
    .driver = {
        // 드라이버의 이름으로 compatible과는 무관함
        .name = "comento-spi",
    },
    .id_table = comento_spi_ids,
    .probe = comento_spi_probe,
    .remove = comento_spi_remove,
};

// 지금 드라이버가 SPI 주변장치를 위한 것임을 명시
module_spi_driver(comento_spi_driver);

MODULE_AUTHOR("DDunAnt<ddunant@comento.com>");
MODULE_DESCRIPTION("Comento SPI device Driver");
MODULE_LICENSE("GPL");
```

<drivers/comento/spi.c>

디바이스 노드 VS sysfs ?

디바이스 드라이버를 사용하기 위해서는 디바이스 노드 사용이 일반적

- Read/write 이외에도 ioctl, mmap 등 다양한 기능 구현 가능
- 디바이스 노드를 생성하기 위한 코드 추가 및 udev 데몬 필요
- Read/write 할 때 copy_from_user / copy_to_user, 현재 위치 업데이트 필요

단순히 문자열을 조회하거나 설정하는 용도라면? Sysfs

- 파일 이름, 권한을 매크로로 한번에 설정 가능
- 단순히 데이터를 주어진 버퍼로 읽거나 복사하기만 하면 됨

sysfs란? 디바이스 드라이버에 대한 정보를 나타내는 목적의 가상 파일 시스템

- ✓ 초급 강의에서 udev 데몬 다룰 때 /sys가 잠시 나왔었죠?
- ✓ 본 강의에서 /sys 밑에 파일을 만드는 법을 다루어 보시다.

드라이버에서 sysfs로 값을 읽고 쓰기

```
static ssize_t comento_show_scale(struct device *dev,
                                struct device_attribute *attr, char *buf)
{ // sysfs 파일을 읽었을 때 호출되는 함수, 문자열을 buf로 출력하기만 하면
  ssize_t ret; // 사용자 공간까지 자동으로 전달됨
  short val;
  struct comento_spi_device *csdev = dev_get_drvdata(dev);
  // 통신하는 동안 또 통신을 시도하면 안되므로 뮤텍스로 보호
  mutex_lock(&csdev->lock);
  csdev->tx_buf[0] = COMENTO_SPI_GET_SCALE; // 송신버퍼에 명령어코드 추가
  // spi_sync라는 API는 메시지에 명시된 대로 SPI 통신을 수행
  ret = spi_sync(csdev->dev, &csdev->msg);
  if (ret < 0) {
    printk(KERN_ERR "comento_spi: spi_sync failed - %d\n", ret);
    mutex_unlock(&csdev->lock);
    return ret;
  } // spi_sync는 SPI 통신을 수행하고 수신버퍼에 데이터를 넣어줌
  // 수신버퍼에 들어간 2byte 데이터를 short 타입으로 변환
  val = (csdev->rx_buf[0] << 8) | csdev->rx_buf[1];
  mutex_unlock(&csdev->lock);
  return sprintf(buf, "%d\n", val); // sprintf로 문자열로 buf에 넣어줌
}

static ssize_t comento_store_zero(struct device *dev,
                                struct device_attribute *attr,
                                const char *buf, size_t len)
{ // sysfs 파일에 쓸 때 호출되는 함수, buf에 써진 내용이 자동으로 전달됨
  ssize_t ret;
  struct comento_spi_device *csdev = dev_get_drvdata(dev);

  if (strcmp(buf, "zero\n")) return len; // "zero\n"라고 썼을때만 동작
  mutex_lock(&csdev->lock);
```

```
csdev->tx_buf[0] = COMENTO_SPI_SET_ZERO; // 송신버퍼에 명령어코드 추가
ret = spi_sync(csdev->dev, &csdev->msg); // spi_sync로 SPI 통신을 수행
if (ret < 0) {
  printk(KERN_ERR "comento_spi: spi_sync failed - %d\n", ret);
  mutex_unlock(&csdev->lock);
  return ret;
}
mutex_unlock(&csdev->lock);
return len;
}

// sysfs로 만들 파일 정의, scale이라는 이름의 664 권한의 파일을 만들
DEVICE_ATTR(scale, 0664, comento_show_scale, comento_store_zero);
static struct attribute *comento_spi_attributes[] = {
  &dev_attr_scale.attr,
  NULL,
};
static const struct attribute_group comento_spi_attr_group = {
  .attrs = comento_spi_attributes,
};

static int comento_spi_probe(struct spi_device *dev)
{
  ... // sysfs 파일 생성
  sysfs_create_group(&dev->dev.kobj, &comento_spi_attr_group);
  return 0;
}
static void comento_spi_remove(struct spi_device *dev)
{
  ... // sysfs 파일 삭제
  sysfs_remove_group(&dev->dev.kobj, &comento_spi_attr_group);
}
```

SPI 드라이버 구현 및 테스트 실습

```
# ls -lah /dev/spidev0*
ls: /dev/spidev0*: No such file or directory
# ls /sys/bus/spi/devices/
spi0.0  spi0.1
# cd /sys/bus/spi/devices/spi0.1/
# ls
driver          of_node          statistics
driver_override power             subsystem
modalias        scale            uevent
# ls -lah scale
-rw-rw-r-- 1 root  root    4.0K Jan  1 00:01 scale
# cat scale
0
```

```
# cat scale
25
# echo "zero" > scale
# cat scale
0
```

```
# cat scale
27
#
```

- 디바이스 트리에서 lwn,bk4를 comento-spi로 변경 해서 SPIDEV가 생기지 않음
- SPI 디바이스는 /sys/bus/spi/devices/에 모여 있음
- SPI0.0이나 SPI0.1을 보면 드라이버에서 만든 scale 파일이 지정한 권한으로 생성되어 있음
- scale의 초기값은 0이므로 sysfs에서도 0을 출력

```
comento@coment:~/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock
machine/peripheral/spi/ssi/child[0].scale 25kg
{}
```

- sysfs파일에 zero를 입력하여 영점 조정 수행

```
comento@coment:~/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock
machine/peripheral/spi/ssi/child[0].scale 52kg
{}
```

SPI 톨로 복잡하게 바이트를 보내고 받을 필요 없이 sysfs로 간편하게 사용



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

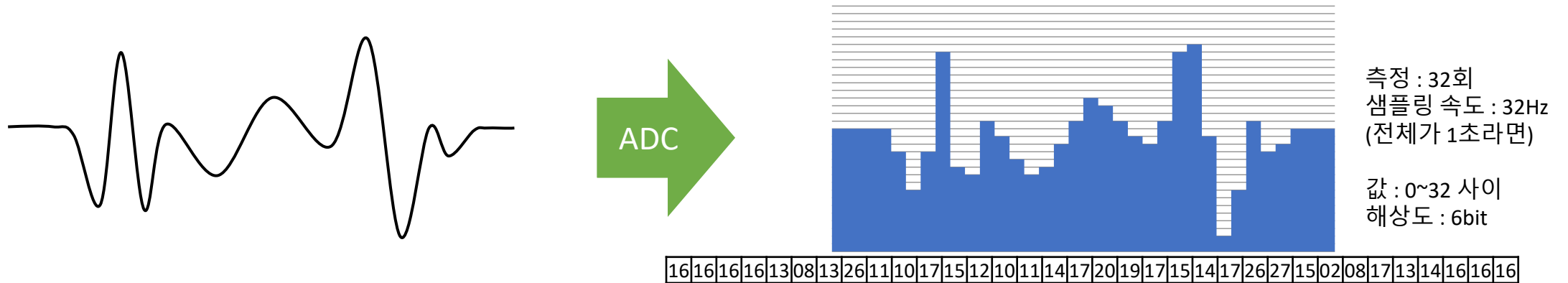
Chapter. 5

ADC 이해하기

ADC (Analog to Digital Converter) 란?

아날로그 신호를 디지털 신호로 변환해 주는 하드웨어

- 아날로그 신호란? 값의 크기(특히, 전압)가 연속적인 값으로 변환하는 신호
- 디지털 신호란? 이진수로 표현할 수 있는 값으로 변환된 불연속적인 신호



왼쪽의 부드러운 아날로그 신호가 오른쪽의 딱딱한 디지털 신호로 바뀜

- 얼마나 빠른 주기 안에 값을 확인하여 측정하느냐 -> 샘플링 속도(Sampling rate)
- 디지털로 표현할 수 있는 단위의 수가 얼마나 되느냐 -> 해상도(Resolution)

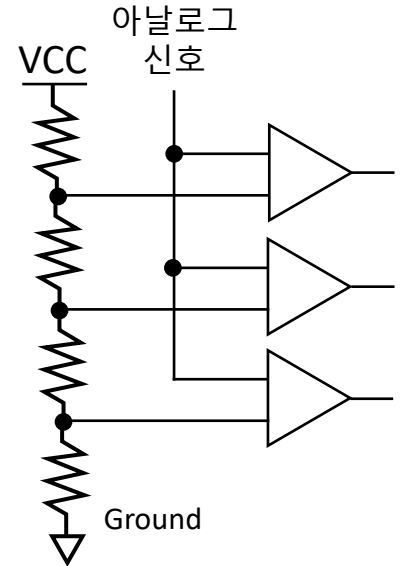
ADC의 구조

OPAMP라는 소자로 만드는 비교기(Comparator)를 사용

- 비교기(Comparator) : 어떤 전압이 기준전압보다 크면 High, 작으면 Low
 - 회로도에서는 일반적으로 삼각형으로 표시

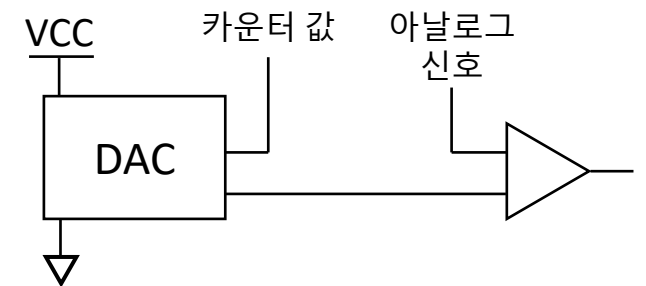
저항을 사용한 전압 분배회로로 비교전압을 여러 개 만들어서 비교

- 비교기를 다수 사용하여야 하므로 단가 상승
- 여러 비교기 각각의 상태가 미세하게 달라 신뢰도가 떨어질 수 있음



DAC를 사용하여 카운터를 통해 비교 전압을 여러 번 만들어 비교

- 비교기를 하나만 사용해도 되므로 소형화 가능
- 미세한 시간차이로 인해 정확도가 떨어짐



ADC 활용 예시

어떤 종류의 아날로그 신호이든 전압 신호로 바꿀 수 있다면 활용 가능

- 음성 신호 : 소리가 마이크를 통해 전압 신호로 바뀜
 - CD급의 음질을 위해서는 16bit 해상도에 44.1kHz 이상의 샘플링 속도가 요구
- 시각 신호 : 각 픽셀이 광자를 받아 전압 신호를 생성함
 - 이미징 센서 내 ADC가 포함되어 있으며 이미징 센서는 픽셀 데이터를 디지털 값으로 전달

각종 센서는 측정하고자 하는 물리적인 세기에 따라 저항이나 전압이 미세하게 바뀜

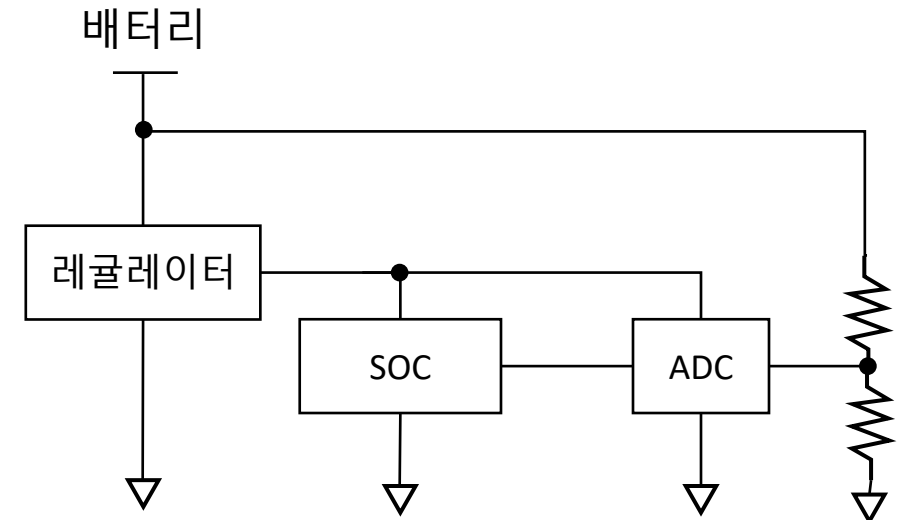
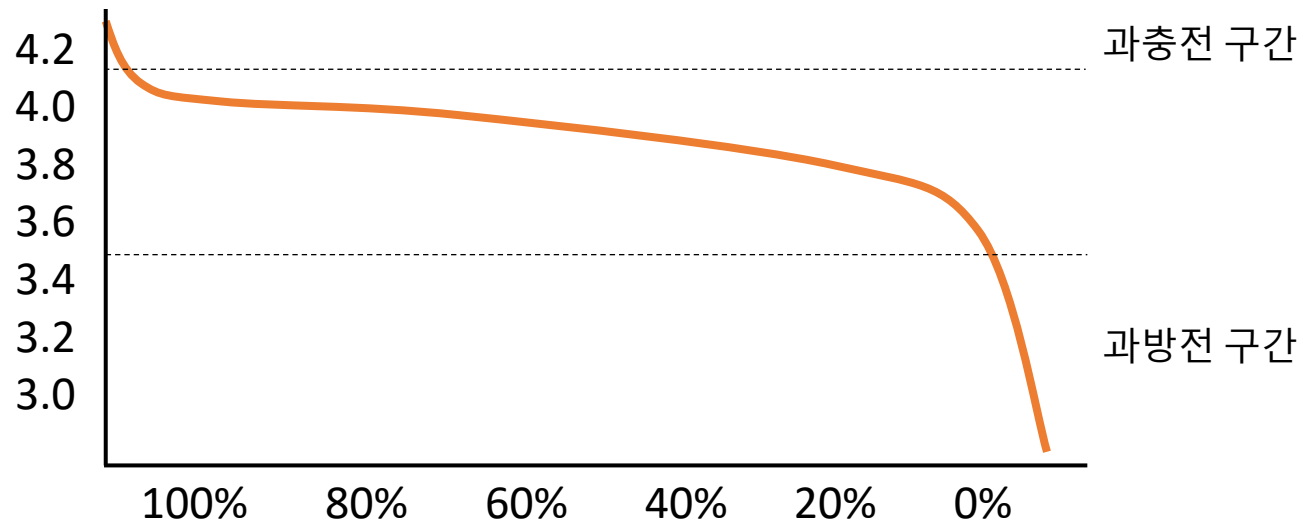
- 높은 수준의 샘플링 속도는 필요하지 않으나 센서에 따라 높은 해상도 요구

➤ 센서에 대해서는 다음 주 수업에서 자세히 다뤄보도록 하겠습니다.

ADC 활용 예시 – 배터리 잔량 체크

배터리 전압 : 완충 때 전압이 가장 높고 잔량에 따라 전압이 점점 감소

- 배터리 전압(3.7V)이 구동 전압(3.3V)보다 큼
 - 레귤레이터를 이용해 전압을 구동 전압으로 맞추어 사용
- 과충전/과방전 될 경우 배터리 수명이 단축
- 전압분배 회로로 전압을 나눈 다음 ADC로 전압 측정, 전압을 통해 잔량 추정



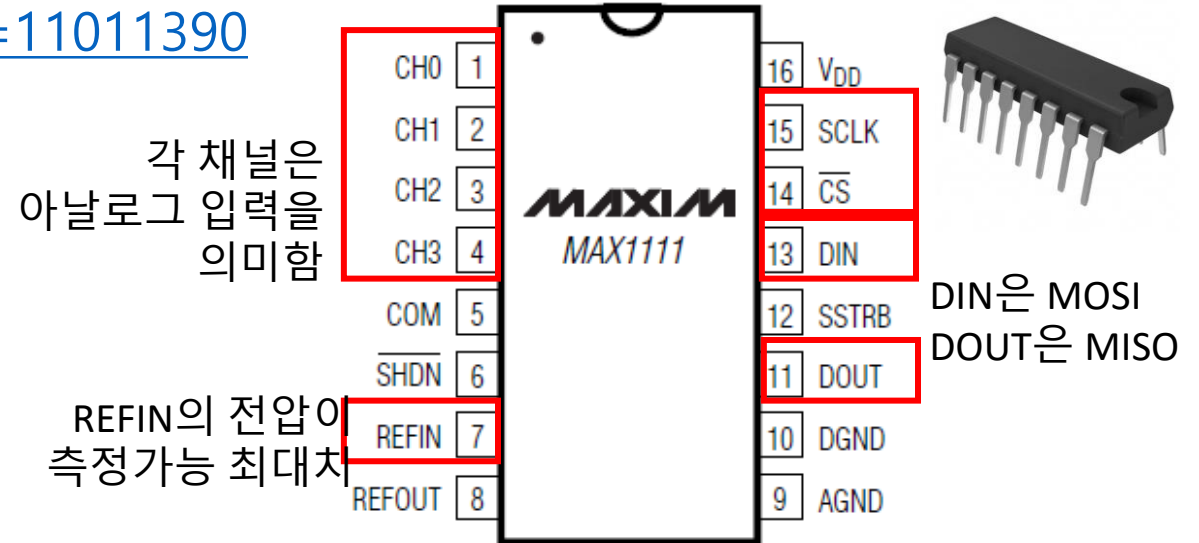
ADC 하드웨어 예시 – MAX1111

MAXIM IC 사의 SPI 통신을 지원하는 저전력 ADC 주변장치

- 해상도 : 8bit, 샘플링 속도 : 50 kHz, 4개의 ADC
- 측정 가능한 전압 범위 : 0 ~ 기준전압(REFIN), 기준전압은 2.7~5.5V 까지 다양하게 가능

<https://www.eleparts.co.kr/goods/view?no=11011390>

- 명령어 코드 1바이트 송신 이후,
반환 값 2바이트 수신



QEMU에 MAX1111 ADC 추가하기

```
static void create_spi_devs(ComentoMachineState *vms) {  
    ...  
    dev = qdev_new("max1111");  
    qdev_prop_set_uint8(dev, "input0", 255); // ADC에서 측정할 값을 설정  
    qdev_prop_set_uint8(dev, "input1", 0);   // 실제 전압이 아닌 8bit 값  
    qdev_prop_set_uint8(dev, "input2", 128);  
    qdev_prop_set_uint8(dev, "input3", 64);  
    ssi_realize_and_unref(dev, ssi, &error_fatal);  
}
```

<QEMU - hw/arm/comento.c>

```
spi@9015000 {  
    ...  
    spidev@2 {  
        compatible = "max1111";  
        spi-max-frequency = <1000000>;  
        reg = <2>;  
    };  
};
```

<리눅스 - arch/arm64/boot/dts/comento/comento.dts>

```
...  
CONFIG_SENSORS_MAX1111
```

<리눅스 - arch/arm64/configs/comento_defconfig>

수정한 defconfig를 적용하기 위해
make comento_defconfig 명령어 사용

MAX 1111 테스트 실습

리눅스 기본 MAX1111 드라이버는 sysfs를 지원

✓ qom-set으로 MAX1111은 중간에 값을 변경할 수 없음 → QEMU 미구현!

```
# cd /sys/bus/spi/devices/spi0.2
# ls
driver      in1_input    name          subsystem
driver_override in2_input    of_node        uevent
hwmon       in3_input    power
in0_input   modalias     statistics
# cat in0_input
2040
# cat in1_input
1024
# cat in2_input
0
# cat in3_input
512
#
```

- spi0.2 디바이스가 리눅스에 탐지
- 드라이버가 sysfs 지원
- 각 ADC 채널에 해당되는 파일 생성
- ADC 채널 파일을 읽으면, 해당 채널에서 측정된 값이 출력
- Sysfs 출력 값 단위는 mV
 - 기준전압이 2048mV라고 가정



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

과제 안내

과제 의도

- SPI 통신이 어떤 식으로 이루어지는지 살펴본다.
- 실제 현업에서 ADC를 어떻게 활용될 수 있는지 생각해본다.

필수 과제 내용

1. 배터리와 연결되어 잔량을 측정하는 SPI 주변 장치를 QEMU에 추가한다.
 1. 배터리의 잔량은 QMP의 percent로 0%~100%까지 설정 가능하게 한다.
 2. 이상한 값을 설정하면 무시한다.
2. sysfs로 percent 파일을 만들어 배터리 잔량을 출력하는 디바이스 드라이버를 만든다.
 1. 읽기만 가능하며 쓰기는 불가능하게 구현한다.
3. QEMU에 추가한 코드와 디바이스 드라이버에서 수정/추가한 코드를 업로드하여 제출한다.

과제 안내

선택 과제 내용

- 필수 과제에서 구현한 드라이버에서 주기적으로 배터리 잔량을 체크하게 한다.
 - 타이머를 통해 주기적으로 함수를 실행하는 예제 코드

```
static void timer_callback(struct timer_list *timer) {  
    // 수행할 작업 작성  
    // 타이머 콜백은 인터럽트 핸들러의 일종이므로,  
    // 오래걸리는 작업이나 시스템 종료 같은 작업은 워크큐를 사용해야 함  
    mod_timer(timer, jiffies + msecs_to_jiffies(2000)); // 2초 뒤로 타이머 재설정  
}  
  
static struct timer_list timer;  
  
static int comento_spi_probe(struct spi_device *dev)  
{  
    ...  
    // timer_callback을 실행하는 타이머 초기화  
    timer_setup(&timer, timer_callback, 0);  
    // 지금 부터 2초 뒤에 타이머 실행되도록 세팅  
    mod_timer(&timer, jiffies + msecs_to_jiffies(2000));  
}
```

과제 안내

선택 과제 내용

2. 배터리 잔량이 5% 미만이면 시스템을 종료한다.

1. 시스템을 종료하는 예제 코드

```
#include <linux/reboot.h>
```

```
kernel_power_off(); // 시스템 종료는 인터럽트 핸들러나 타이머 콜백에서 직접 호출 불가
```

3. 작성한 드라이버 코드를 업로드하여 제출한다.

기한 : 다음 주 월요일 오전 0시까지

다음 주 커리큘럼

I2C 통신 이해하기

새로운 I2C 하드웨어 만들기

I2C 툴로 I2C 하드웨어 테스트하기

I2C 하드웨어 드라이버 만들기

자주 사용되는 센서 파악하기

